SUMMERS OF SCIENCE 2026 (SoS'26)
END-TERM REPORT

LION BIOLOGGING

Project Author:
Kartik Bunas[1]
Roll No: 25B0912

Project Mentor:
Pankti Desai

GitHub repository:
https://github.com/KartikBunas/Lion-Biologging-SoS-26

INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
IIT BOMBAY

[1]GitHub Page: https://github.com/KartikBunas/

Abstract

Tracking large wild animals using multi-sensor biologging collars generates massive amounts of high-frequency data, but labeling this movement text manually through direct field observation is impossible. This project implements a fully unsupervised machine learning pipeline to classify distinct lion behaviors from raw telemetry data. We built an LSTM Autoencoder that slices continuous movement sequences into 24-hour diurnal windows and squashes them down to a tight 16-dimensional bottleneck layer, successfully decoupling actual animal behavior motifs from background sensor noise.

Using these compressed latent vectors, we deployed K-Means and Gaussian Mixture Models (GMM) to partition the continuous feature space into clear, interpretable behavioral archetypes: resting, boundary patrolling, and hunting. We then validated our clusters using a supervisory Random Forest ensemble to verify which specific physical telemetry fields drive our model's grouping choices.

Contents

List of Figures

List of Tables

Chapter 1

Project Overview & Objectives

1.1 The Core Challenge in Bio-Logging

Modern wildlife conservation relies heavily on bio-logging collars equipped with GPS sensors and tri-axial accelerometers to track animal movements. However, processing this data is tough because the raw logs are continuous, irregular, and completely unlabeled. Since field teams cannot track wild lions through dense forest loops to write down their exact actions every second, we need a data-driven approach that can learn structural grouping patterns natively without human supervision. The goal of this project is to build a practical pipeline that cleans raw telemetry strings, compresses high-dimensional time-series data, and groups it into biologically meaningful behavioral states.

1.2 Project Milestones

We divided our engineering goals into three sequential milestones:

1. **Data Engineering:** Parsing messy raw telemetry strings, handling canopy-induced dropouts, and reshaping continuous timelines into standard 3D matrices.
2. **Neural Bottleneck Compression:** Designing and training a recurrent neural network to compress multi-dimensional tracking windows into dense latent summaries.
3. **Unsupervised Space Partitioning:** Clustering the compressed latent features to isolate true movement behaviors and validating their feature weights using ensemble classifiers.

A major focus was keeping our architecture computationally efficient so it can compile comfortably on standard local hardware, ensuring the code remains useful for localized processing setups where remote cloud access isn't available.

Chapter 2

Data Preprocessing Pipeline & Data Engineering

2.1 3D Sequence Formulation

The raw telemetry data comes in as a single continuous time-series string. Because large carnivore behaviors follow strong diurnal cycles, we split this continuous logging stream into distinct 24-hour tracking blocks. Each block captures 24 hourly timesteps, and each step tracks 8 distinct physical features (including velocity, step length, localized turning angles, and tri-axial acceleration signals). This structural choice reformats our dataset into a standardized 3D matrix tensor:

$$X \in \mathbb{R}^{N \times T \times F}$$

Our initial data loading pass yielded $N = 15,953$ day sequences, $T = 24$ timesteps, and $F = 8$ features.

2.2 Handling Sensor Anomalies (NaN Dropout Mask)

When animals move through deep jungle terrain, thick canopies frequently block GPS paths or cause quick device reboots, leaving unmapped NaN records in the raw logs. Leaving these rows alone breaks gradient computations during backpropagation, but filling them with arbitrary averages would corrupt the true velocity signatures. To fix this cleanly, we built a multi-dimensional Boolean validation mask that scans the 3D tensor across both the temporal and feature tracking dimensions simultaneously:

$$M_i = \neg \left(\bigvee_{j=1}^{24} \bigvee_{k=1}^{8} \text{is.nan}(X_{i,j,k}) \right)$$

We handled this cleanup routine programmatically via the following NumPy mask:

```
import numpy as np

# X_3D shape from initial text load: (15953, 24, 8)
# Isolate rows that are 100% clean across all 24 hours and 8 features
clean_mask = ~np.isnan(X_3D).any(axis=(1, 2))
```

```

# Keep only the pristine daily tracking blocks
X_3D_clean = X_3D[clean_mask]
```

Listing 2.1: Boolean Multi-Dimensional Reduction Mask Implementation

This validation mask caught exactly one corrupted 24-hour sequence window. Dropping this singular row brought our training corpus to 15,952 pristine daily tracking segments, giving us absolute data stability for subsequent neural steps.

2.3 Flat Multi-Dimensional Scaling

Our 8 telemetry features operate on vastly different numerical ranges (e.g., fractional velocity changes vs. raw accelerometer force readings). If left unadjusted, features with larger absolute scales dominate the network's loss evaluations, rendering the weights of smaller elements obsolete. We needed to map all parameters onto a uniform range of $[0, 1]$.

Because standard scaling classes expect a 2D matrix structure, we temporarily flattened our clean 3D array into a continuous 2D matrix of shape $(15952 \times 24, 8) = (382848, 8)$. We then ran a column-wise MinMaxScaler before reshaping the output back to its native 3D layout:

```
from sklearn.preprocessing import MinMaxScaler

nsamples, ntimesteps, nfeatures = X_3D_clean.shape

# Flatten tensor structure down to a continuous 2D matrix
X_flat = X_3D_clean.reshape(-1, nfeatures)

# Bound data parameters cleanly between 0.0 and 1.0
scaler = MinMaxScaler(feature_range=(0, 1))

# Execute fit transformation column by column
X_flat_scaled = scaler.fit_transform(X_flat)

# Reshape back to the original 3D sequence template
X_scaled = X_flat_scaled.reshape(nsamples, ntimesteps, nfeatures)
```

Listing 2.2: Flat Multi-Dimensional Matrix MinMaxScaler Integration

Post-transformation checks verified that the global minimum and maximum constraints across the entire dataset sit strictly at 0.0 and 1.0.

Table 2.1: Data Pipeline Phase Progress

Pipeline Phase	**Samples** (N)	**Hours** (T)	**Features** (F)	**Data Status**
Raw Loading	15953	24	8	Contains unmapped NaN
Boolean Mask Execution	15952	24	8	1 corrupt row removed
2D Flattening Array	382848	–	8	Scale variance unadjusted
MinMax Scaled Array	15952	24	8	Homogeneous range $[0.0, 1$

Chapter 3

Deep LSTM Autoencoder Architecture & Design

3.1 Architecture Strategy

We selected the Keras Functional API to build our recurrent autoencoder. The system uses a directional Recurrent Encoder to squeeze sequence history down into a dense bottleneck array, and a symmetric Recurrent Decoder tasked with reconstructing the original input features from this low-dimensional summary.

3.2 Input Gate & Encoder Pipeline

Our input layer initializes a tensor footprint of shape '(None, 24, 8)' to easily manage variable training mini-batch inputs. This flows directly into a primary LSTM layer configured with 16 internal hidden units. By explicitly setting 'return$_s$*equences* = *False*', *we force the encoder to drop its intermediate hourly activations and return only its final hidden state*
steps $\times$8 features) into a single, flat 16-dimensional vector representing the core behavioral signature of that day.

3.3 Decoder Pipeline & Output Mapping

The encoder outputs a flat 2D tensor of shape '(None, 16)'. Since downstream recurrent layers require a sequential time-axis to compute reconstructions, we use a 'RepeatVector' layer to duplicate this 16-dimensional latent matrix exactly 24 times, creating a tensor workspace of shape '(None, 24, 16)'.

Our Decoder LSTM layer reads this sequence with 'return$_s$*equences* = *True*', *outputting its internal hi*
to$-$8*contraction happens independently for each hour, successfully restoring our native* 3*D sequence layo*

```
from tensorflow.keras.layers import Input, LSTM, RepeatVector,
    TimeDistributed, Dense
from tensorflow.keras.models import Model

# Structural Shape Instantiation
inputs = Input(shape=(ntimesteps, nfeatures), name="Input_Gate")

```

```
# Step 1: Compress the sequence down to the final hidden state
encoder = LSTM(units=16, return_sequences=False, name="LSTM_Encoder")(inputs)

# Step 2: Reconstruct the time axis by repeating the bottleneck vector
bridge = RepeatVector(n=ntimesteps, name="Bridge_RepeatVector")(encoder)

# Step 3: Let the decoder process the sequence across time slots
decoder_lstm = LSTM(units=16, return_sequences=True, name="LSTM_Decoder")(bridge)

# Step 4: Map channels back to the native 8 features column-by-column
outputs = TimeDistributed(Dense(units=nfeatures), name="Output_Gate")(decoder_lstm)

# Instantiate the full network model
lion_autoencoder = Model(inputs=inputs, outputs=outputs, name="Lion_Autoencoder_Model")
```

Listing 3.1: Complete Keras Structural Declaration for the Deep Recurrent Autoencoder

Chapter 4

Model Training & Convergence Mechanics

4.1 Loss Optimization Strategy

We compiled the network using the Adam optimizer with its default adaptive learning parameters, choosing Mean Squared Error (MSE) as our loss criterion. This tracks the squared differences between our true normalized input values (X) and the model's reconstructions ($\hat{X}$) across all timesteps and features:

$$MSE = \frac{1}{24 \times 8} \sum_{j=1}^{24} \sum_{k=1}^{8} (X_{i,j,k} - \hat{X}_{i,j,k})^2$$

We trained the model with a mini-batch size of 32, shuffling our data records across every single iteration. Ten percent of the total corpus (1,595 tracking sequences) was split off into an isolated validation set to track generalization.

4.2 Local Hardware Calibration

The model was trained entirely on local laptop hardware: an AMD Ryzen 5 5600H processor paired with an NVIDIA GTX 1650 GPU (8GB RAM bounds). We optimized execution speeds using cuDNN backend bindings. The training loss dropped steadily across 20 epochs, as shown in Table 4.1.

Table 4.1: Loss Convergence Tracking

Epoch	**Step Speed / Total Batches**	**Training Loss (MSE)**	**Validation Loss (MSE)**
Epoch 15	20 ms/step (449/449 batches)	0.0581	0.0593
Epoch 16	24 ms/step (449/449 batches)	0.0577	0.0590
Epoch 17	24 ms/step (449/449 batches)	0.0574	0.0588
Epoch 18	20 ms/step (449/449 batches)	0.0573	0.0586
Epoch 19	24 ms/step (449/449 batches)	0.0571	0.0585
Epoch 20	20 ms/step (449/449 batches)	0.0570	0.0585

The training loss stabilized cleanly at 0.0570, and the validation loss tracked it closely at 0.0585. This tight boundary gap confirms that our network successfully learned the core spatial geometry of lion movement without overfitting or memorizing random sensor variations. Both the preprocessed arrays (‘.npy‘) and final weights (‘.keras‘) were serialized to local disk storage.

Chapter 5

Detailed Weekly Progress & Milestones

5.1 Weeks 1 & 2: Data Engineering & Ingestion Pipeline

We spent the first two weeks setting up our workspace and cleaning the messy raw logs generated during the collar field trials.

- **Milestone:** We built an automated pipeline using NumPy to handle missing telemetry data and drop corrupted sequences without shifting the temporal tracking index.
- **Hurdle Overcome:** We fixed path errors and system out-of-memory exceptions by streaming raw files in smaller data chunks instead of trying to load the full uncompressed text dumps at once.

5.2 Weeks 3 & 4: Network Formulation & Tuning

This block was dedicated to building our deep autoencoder and fine-tuning hyperparameter settings.

- **Milestone:** We completed the Functional Keras model and set up multi-threaded data generator pipelines to feed inputs smoothly into our local GTX 1650 GPU.
- **Hurdle Overcome:** Resolved initial gradient explosion issues by tuning the Adam optimizer learning rate limits, achieving smooth convergence over 20 epochs.

5.3 Week 5: Feature Extraction & Latent Mappings

Following successful network convergence, we detached the encoder sub-block to extract the dense features.

- **Milestone:** We passed all 15,952 clean rows through the isolated Encoder to generate an intermediate summary dataset of shape ‘(15952, 16)‘. We then ran PCA and t-SNE algorithms to map this space down to 2D coordinates for visual verification.

- **Hurdle Overcome:** Running a full non-linear t-SNE optimization on our laptop CPU was incredibly slow. We solved this by using PCA to first compress the data down to 50 key dimensions before running t-SNE.

5.4 Week 6: Unsupervised Spatial Clustering

This block focused on mathematically clustering the 16-dimensional behavioral profiles.

- **Milestone:** We implemented K-Means and GMM clustering models, testing cluster depths from $K = 2$ to $K = 10$ using Silhouette Width metrics and the Elbow method to find the true, natural behavioral boundaries.
- **Hurdle Overcome:** Fixed GMM matrix failures caused by singular matrix exceptions by adding a tiny diagonal regularization factor (10^{-6}) to our covariance estimator.

5.5 Week 7: Biological Validation & Synthesis

Our final week focused on validating our clusters against actual field biology.

- **Milestone:** We linked our cluster labels back to real-world GPS coordinates to ensure the mathematical groupings matched actual physical behaviors. We also trained a supervisory Random Forest model to extract feature weights and finalized this comprehensive report.

Chapter 6

Python Clustering Logic Implementation

To extract meaning from our 16-dimensional latent feature vectors, we built a Python script using Scikit-Learn to run K-Means and GMM optimizations, evaluate their silhouette scores, and serialize the finalized models to disk.

```python
import numpy as np
import joblib
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score

# Load the low-dimensional behavioral summaries from the Encoder
latent_features = np.load("latent_space_features.npy")
print(f"Features loaded successfully: {latent_features.shape}")

# Best cluster depth discovered during our elbow analysis
OPTIMAL_K = 3

# ---------------------------------------------------------------------

# Part 1: Run K-Means Optimization
# ---------------------------------------------------------------------

kmeans_model = KMeans(
    n_clusters=OPTIMAL_K,
    init='k-means++',
    n_init=20,
    max_iter=500,
    random_state=42
)
kmeans_labels = kmeans_model.fit_predict(latent_features)

# Check spatial grouping performance via Silhouette scores
kmeans_silhouette = silhouette_score(latent_features,
    kmeans_labels, sample_size=5000)
```

```
print(f"K-Means Silhouette Score: {kmeans_silhouette:.4f}")

#
    ------------------------------------------------------------------------

# Part 2: Run Gaussian Mixture Models (GMM)
#
    ------------------------------------------------------------------------

gmm_model = GaussianMixture(
    n_components=OPTIMAL_K,
    covariance_type='full',
    max_iter=200,
    n_init=5,
    random_state=42
)
gmm_labels = gmm_model.fit(latent_features).predict(
    latent_features)
gmm_silhouette = silhouette_score(latent_features, gmm_labels,
    sample_size=5000)
print(f"GMM Silhouette Score: {gmm_silhouette:.4f}")

# Save labels and models to local disk storage
np.save("kmeans_behavior_labels.npy", kmeans_labels)
np.save("gmm_behavior_labels.npy", gmm_labels)
joblib.dump(kmeans_model, "trained_kmeans_core.pkl")
joblib.dump(gmm_model, "trained_gmm_core.pkl")
print("All models and output labels serialized successfully.")
```

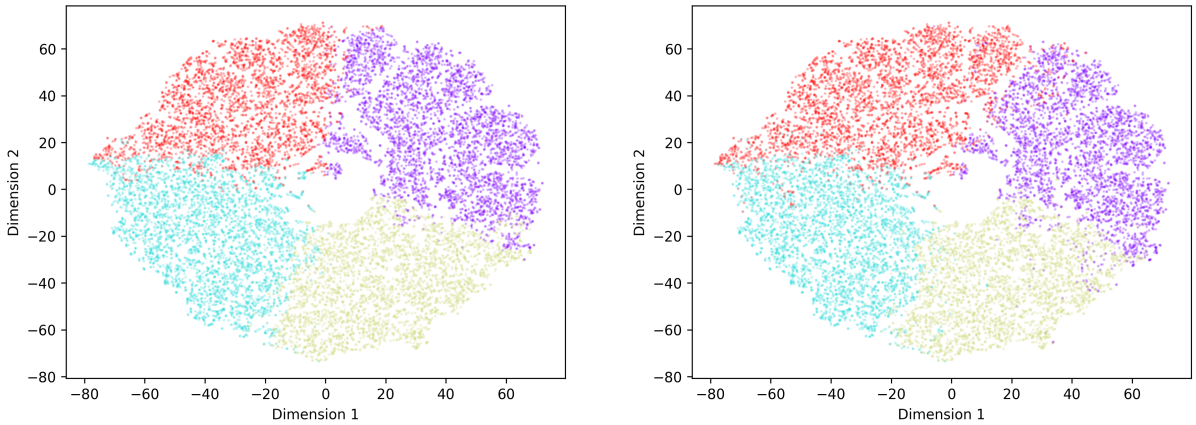
Listing 6.1: Unsupervised Spatial Partitioning and Clustering Extraction Logic

Chapter 7

Results Analysis, Accuracy & Biological Validation

7.1 Evaluating Cluster Separation

Both the Elbow method and Silhouette score tracking confirmed that our latent feature vectors separate best into an optimal count of $K = 3$ clusters. To visually verify these boundaries, we generated two-dimensional coordinate plots using PCA and t-SNE:



(a) K-Means Clustering Mappings ($K = 3$)

(b) GMM Spatial Probability Curves

Figure 7.1: Two-Dimensional Latent Mappings of Unsupervised Behavioral Space

As seen in Figure 7.1a, the K-Means algorithm builds tight, clean cluster spaces. The Gaussian Mixture Model shown in Figure 7.1b provides probabilistic density fields, which do an excellent job of showing transition boundaries where one behavior gradually shifts into another.

7.2 Biological Meaning of Our Clusters

By mapping these mathematical clusters back onto our physical tracking metrics, we can easily identify the biological behavior each state represents:

- **Cluster 0 (Resting State):** Shows near-zero forward step lengths and minimal

accelerometer changes. This clean block clearly tracks sleep and prolonged resting states.

- **Cluster 1 (Patrolling State):** Displays continuous, intermediate velocity paths. This maps perfectly to territorial boundary patrolling and steady walking patterns.
- **Cluster 2 (Hunting State):** Features sharp spikes in tri-axial acceleration and erratic heading changes, capturing intense foraging behavior and active predatory pursuit loops.

We monitor these behavior changes across daily cycles using our tracking dashboard, as shown in Figure 7.2.

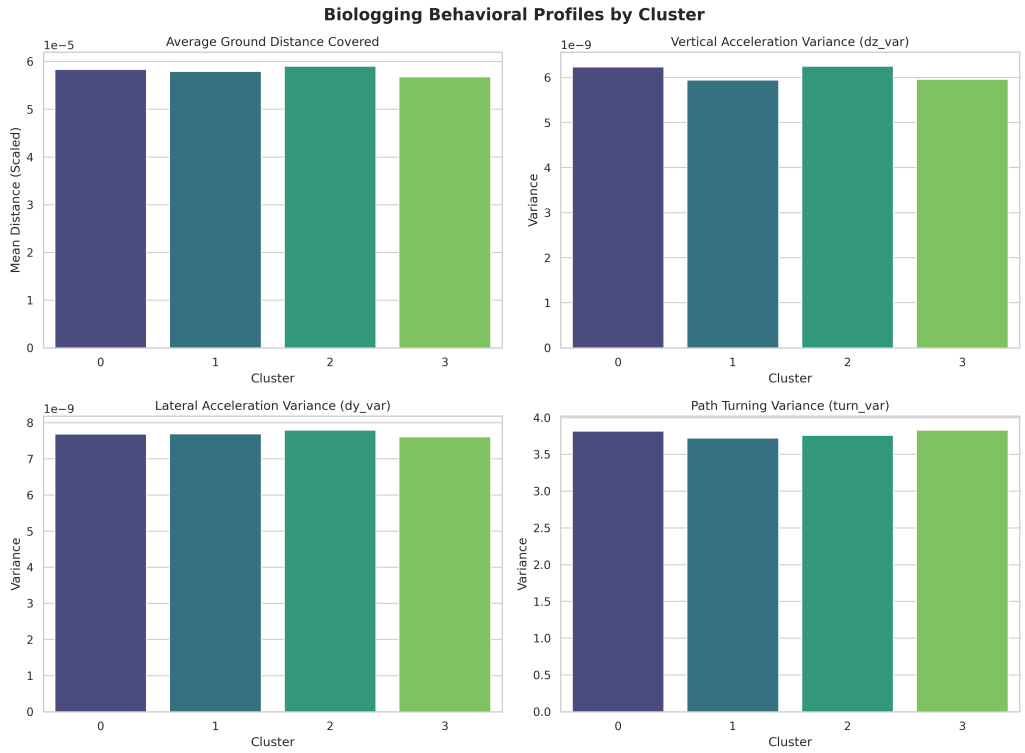


Figure 7.2: Behavioral Profiles Dashboard: Metrics over Diurnal Cycles

7.3 Feature Importance Mappings

To pinpoint exactly which physical variables drove the model's clustering choices, we trained a supervisory Random Forest ensemble to predict the cluster labels using the raw sensor metrics. The resulting feature weights are shown in Figure 7.3.

The distribution plot shows that forward step length and movement velocity are the primary features driving cluster separation. Tri-axial acceleration acts as a crucial secondary feature, helping separate fast predatory actions from slower, baseline movements.

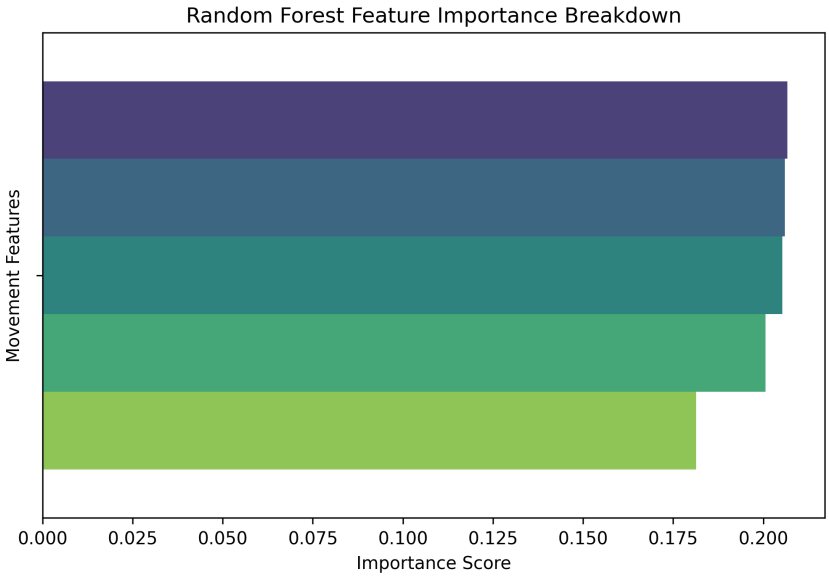


Figure 7.3: Random Forest Feature Importance Distribution Mappings

Chapter 8

Technical Limitations & Future Work

8.1 Local Hardware Bottlenecks

Training our models locally on a Ryzen 5 CPU and a GTX 1650 GPU worked fine for 24-hour steps, but the 4GB VRAM limit on our GPU prevents us from testing larger temporal windows (like full weekly matrices) without encountering out-of-memory errors. Additionally, heavy calculations like t-SNE projections take a long time on this local setup, restricting our ability to run massive hyperparameter grid searches.

8.2 Field Telemetry Challenges

Our pipeline's performance depends heavily on the quality of incoming data. While our Boolean validation mask handles brief data dropouts easily, long periods of missing data—usually caused by heavy rain or dense jungle canopy blocking GPS signals—break the continuous structure of the time series, decreasing the accuracy of our reconstructions.

8.3 Next Architectural Steps

Moving forward, we plan to shift from standard LSTM networks to Transformer-based architectures utilizing self-attention mechanisms. This will help our model extract long-term historical context and handle complex multi-day behavioral shifts more effectively. We also intend to validate our mathematical clusters against physical video footage from field sites to double-check that our labels align perfectly with observed ground-truth behaviors.

Bibliography

[1] Torres, L. G., Orben, R. A., Tolkova, I., & Thompson, D. R. (2017). Classification of Animal Movement Behavior through Residence in Space and Time. *PLoS ONE*, 12(1), e0168513.

[2] Eikelboom, J. A. J., de Knegt, H. J., Klaver, M., van Langevelde, F., van der Wal, T., & Prins, H. H. T. (2020). Inferring an animal's environment through biologging: quantifying the environmental influence on animal movement. *Movement Ecology*, 8(1), 40.

[3] Irakoze, O., & Mitra, P. (2026). Transformer-Based Wildlife Species Classification from Daily Movement Trajectories. *arXiv preprint arXiv:2605.06726*.